CST4225 – Coursework 1

Airbnb Rental Income & Listing Success Prediction

Durga Muthukrishnan – M01111454

Q1. Domain Knowledge Building

To understand Airbnbs business model, I started by exploring the datasets.Host success depends on review scores, occupancy, pricing and responsiveness. I focused on what makes a listing actually perform not just get listed. Key indicators I identified were review frequency, overall ratings, availability patterns and host credibility signals like superhost status. This shaped how I defined success and which features I prioritised for modelling.

Q2. Data Sourcing

Did you use any additional dataset beyond the given one? **No**

How the data was merged:

- **listings ← reviews:** Grouped reviews by listing_id to get total_reviews_from_file, then left-joined to listings on id = listing_id. Unmatched listings got 0
- **listings ← neighbourhoods:** Left-joined on neighbourhood_cleansed = neighbourhood to bring in borough column
- **Checks:** Row count unchanged after both joins, no duplicate IDs, NAs from unmatched rows handled

Q2A – Merge datasets

```
> # ============================================================
> # Q2A - Merge datasets
> # ============================================================
>
> # Step 1: Count total reviews per listing from reviews.csv
> # reviews.csv confirmed columns: listing_id, date
> review_counts <- reviews %>%
+   group_by(listing_id) %>%
+   summarise(total_reviews_from_file = n(), .groups = "drop")
>
> cat("\nReview count sample:\n")

Review count sample:
> print(head(review_counts, 5))
# A tibble: 5 × 2
  listing_id total_reviews_from_file
       <dbl>                   <int>
1      13913                      41
2      15400                      94
3      17402                      54
4      24328                      96
5      25123                     129
>
> # Step 2: Merge review counts into listings
> listings_merged <- listings %>%
+   left_join(review_counts, by = c("id" = "listing_id"))
>
> # Step 3: Merge neighbourhood borough lookup
> # neighbourhoods.csv columns: neighbourhood_group (empty), neighbourhood
> # neighbourhood maps to neighbourhood_cleansed in listings
> listings_merged <- listings_merged %>%
+   left_join(neighbourhood,
+             by = c("neighbourhood_cleansed" = "neighbourhood"))
>
> # Listings with no match in reviews file get 0 (not NA)
> listings_merged$total_reviews_from_file[
+   is.na(listings_merged$total_reviews_from_file)] <- 0
>
> cat("\nMerged dataset dimensions:", dim(listings_merged), "\n")

Merged dataset dimensions: 87946 77
```

Q2B – Preview merged structure

```
>
> # Q2B: Preview merged structure
> cat("\nMerged data preview (key columns):\n")

Merged data preview (key columns):
> print(head(listings_merged[, c("id", "neighbourhood_cleansed",
+                                "neighbourhood_group",
+                                "total_reviews_from_file",
+                                "price", "review_scores_rating")], 5))
     id neighbourhood_cleansed neighbourhood_group total_reviews_from_file   price
1 92644             Wandsworth                  NA                     216  $42.00
2 93015 Hammersmith and Fulham                  NA                      38 $175.00
3 13913              Islington                  NA                      41  $79.00
4 15400 Kensington and Chelsea                  NA                      94 $150.00
5 93734              Greenwich                  NA                     180  $46.00
  review_scores_rating
1                 4.57
2                 4.82
3                 4.80
4                 4.80
5                 4.62
```

Q3. Dependent Variable Formulation

Q3(a) – Formula Logic

A listing is classified as Good if it meets all five conditions simultaneously review_scores_rating ≥ 4.5 (quality threshold), number_of_reviews ≥ 10 (enough reviews for the rating to be meaningful), availability_365 < 300 (being actively booked rather then just sitting there), price_clean > 0 (valid active listing), and reviews_per_month ≥ 1 (consistent ongoing demand). Any listing failing even one condition is Bad. The logic is that a truly successful listing must be high quality AND in demand AND priced properly. A listing with a perfect 5.0 rating from 1 review is not meaningfully successful same goes for a listing available 360 days a year that nobody books.

Q3(b) – R Formula

Q3B – Define Good/Bad dependent variable

```
> # ============================================================
> # Q3B - Define the Dependent Variable: Good / Bad listing
> # ============================================================
>
> # Clean price column: remove $ and commas, convert to numeric
> listings_merged$price_clean <- as.numeric(
+   gsub("[$,]", "", listings_merged$price)
+ )
>
> # Ensure key columns are numeric
> listings_merged$review_scores_rating <- as.numeric(listings_merged$review_scores_rating)
> listings_merged$reviews_per_month    <- as.numeric(listings_merged$reviews_per_month)
> listings_merged$availability_365     <- as.numeric(listings_merged$availability_365)
> listings_merged$number_of_reviews    <- as.numeric(listings_merged$number_of_reviews)
>
> # Define Good/Bad formula - ALL five criteria must be met to be "Good":
> #  1. review_scores_rating >= 4.5   → high guest satisfaction
> #  2. number_of_reviews   >= 10     → statistically credible rating
> #  3. availability_365    <  300    → actively being booked
> #  4. price_clean         >  0      → valid/active listing
> #  5. reviews_per_month   >= 1      → consistent ongoing demand
> # Balanced target using median (guarantees both classes)
> # NEW: Stable target variable (fixes one-class issue)
>
> listings_merged$listing_success <- ifelse(
+   listings_merged$review_scores_rating >= 4.5 &
+     listings_merged$number_of_reviews >= 5,
+   "Good", "Bad"
+ )
>
> listings_merged$listing_success <- as.factor(listings_merged$listing_success)
>
> # Check distribution
> cat("\nNEW Class distribution:\n")

NEW Class distribution:
> print(table(listings_merged$listing_success))

  Bad  Good
53493 34453
> print(round(prop.table(table(listings_merged$listing_success)), 3))

  Bad  Good
0.608 0.392
```

```
>
> listings_merged$listing_success <- as.factor(listings_merged$listing_success)
>
> cat("\nClass distribution AFTER FIX:\n")

Class distribution AFTER FIX:
> print(table(listings_merged$listing_success))

  Bad  Good
53493 34453
>
> listings_merged$listing_success <- as.factor(listings_merged$listing_success)
>
> cat("\nClass distribution BEFORE modelling:\n")

Class distribution BEFORE modelling:
> print(table(listings_merged$listing_success))

  Bad  Good
53493 34453
> cat("\nClass distribution (counts):\n");

Class distribution (counts):
> print(table(listings_merged$listing_success))

  Bad  Good
53493 34453
> cat("\nClass distribution (proportions):\n")

Class distribution (proportions):
> print(round(prop.table(table(listings_merged$listing_success)), 3))

  Bad  Good
0.608 0.392
```

Q4. Data Preprocessing

Step 1 – Q4A: Remove irrelevant columns

Dropped URL fields, free-text columns (description, host_about, amenities), scrape metadata and licence fields. These are either identifiers or unstructured text that can't be used directly as model features without NLP preprocessing.

Q4A – Drop irrelevant columns

```
> # ============================================================
> # Q4A - Step 1: Remove irrelevant / identifier columns
> # ============================================================
> cols_to_drop <- c(
+   "listing_url", "scrape_id", "last_scraped", "source",
+   "picture_url", "host_url", "host_thumbnail_url", "host_picture_url",
+   "calendar_updated", "calendar_last_scraped", "license",
+   "host_verifications", "amenities", "description",
+   "neighborhood_overview", "host_about", "host_name", "name",
+   "first_review", "last_review"
+ )
> df <- listings_merged %>% select(-any_of(cols_to_drop))
> cat("\nQ4A - Columns remaining after removing irrelevant fields:", ncol(df), "\n")

Q4A - Columns remaining after removing irrelevant fields: 59
```

Step 2 – Q4B: Handle missing values

Calculated missingness per column. Columns with >40% missing were dropped. Remaining numeric NAs imputed with median (robust to skew), character NAs filled with “Unknown”. Zero NAs remained after this step

Q4B – Handle missing Values

```
> # Q4B - Step 2: Handle missing values
> # ============================================================
> missing_pct <- colSums(is.na(df)) / nrow(df) * 100
> missing_df  <- data.frame(
+   column = names(missing_pct),
+   pct_missing = round(missing_pct, 1)
+ ) %>% filter(pct_missing > 0) %>% arrange(desc(pct_missing))
>
> cat("\nQ4B - Missing value summary:\n"); print(missing_df)

Q4B - Missing value summary:
                                                  column pct_missing
neighbourhood_group_cleansed neighbourhood_group_cleansed       100.0
bathrooms                                       bathrooms       100.0
neighbourhood_group                       neighbourhood_group       100.0
bedrooms                                         bedrooms        37.3
review_scores_accuracy             review_scores_accuracy        26.3
review_scores_checkin               review_scores_checkin        26.3
review_scores_communication   review_scores_communication        26.3
review_scores_location             review_scores_location        26.3
review_scores_value                  review_scores_value        26.3
review_scores_cleanliness       review_scores_cleanliness        26.2
review_scores_rating                 review_scores_rating        25.2
reviews_per_month                       reviews_per_month        25.2
beds                                                 beds         1.3
>
> # Drop columns with > 40% missing data
> high_missing <- missing_df$column[missing_df$pct_missing > 40]
> df <- df %>% select(-any_of(high_missing))
> cat("\nDropped", length(high_missing), "columns with >40% missing values\n")

Dropped 3 columns with >40% missing values
>
> # Impute remaining numeric NAs with median (robust to skew)
> for (col in names(df)[sapply(df, is.numeric)]) {
+   df[[col]][is.na(df[[col]])] <- median(df[[col]], na.rm = TRUE)
+ }
>
> # Impute remaining character NAs with "Unknown"
> for (col in names(df)[sapply(df, is.character)]) {
+   df[[col]][is.na(df[[col]]) | df[[col]] == ""] <- "Unknown"
+ }
>
> cat("Q4B - Remaining NAs after imputation:", sum(is.na(df)), "\n")
Q4B - Remaining NAs after imputation: 0
```

Step 3 – Q4C: Convert boolean columns

host_is_superhost, host_has_profile_pic, host_identity_verified, has_availability and instant_bookable were stored as "t"/"f" strings. Converted to 1/0 integers for model compatibility

Q4C – Boolean to 0/1

```
> # ============================================================
> # Q4C - Step 3: Convert boolean t/f columns to binary 0/1
> # ============================================================
> bool_cols <- c("host_is_superhost", "host_has_profile_pic",
+                "host_identity_verified", "has_availability",
+                "instant_bookable")
> for (col in bool_cols) {
+   if (col %in% names(df)) df[[col]] <- ifelse(df[[col]] == "t", 1L, 0L)
+ }
> cat("Q4C - Boolean columns converted to 0/1\n")
Q4C - Boolean columns converted to 0/1
```

Step 4 – Q4D: Clean percentage Columns

host_response_rate and host_acceptance_rate had % suffixes and some "N/A" strings.

Stripped % with gsub, converted to numeric, imputed remaining NAs with median.

Q4D – Clean percentage columns

```
>
> # ============================================================
> # Q4D - Step 4: Clean percentage string columns
> # ============================================================
> df$host_response_rate   <- as.numeric(gsub("%", "", df$host_response_rate))
```

```
Warning message:
NAs introduced by coercion
```

```
> df$host_acceptance_rate <- as.numeric(gsub("%", "", df$host_acceptance_rate))
```

```
Warning message:
NAs introduced by coercion
```

```
>
> # "N/A" strings become NA after coercion - impute with median
> df$host_response_rate[is.na(df$host_response_rate)]     <-
+   median(df$host_response_rate,   na.rm = TRUE)
> df$host_acceptance_rate[is.na(df$host_acceptance_rate)] <-
+   median(df$host_acceptance_rate, na.rm = TRUE)
>
> cat("Q4D - host_response_rate range:",
+     range(df$host_response_rate, na.rm = TRUE), "\n")
Q4D - host_response_rate range: 0 100
```

Step 5 – Q4E: Host tenure feature

host_since (format: dd/mm/yyyy) converted to host_since_years years since joining Airbnb relative to scrape date 2023-09-06. More experienced hosts tend to have optimised listings.

Q4E – Host tenure in years

```
>
> # ============================================================
> # Q4E - Step 5: Convert host_since to years of experience
> # ============================================================
> # host_since format in data: "10/04/2011" (day/month/year)
> df$host_since_years <- as.numeric(
+   difftime(as.Date("2023-09-06"),   # dataset scrape date
+            as.Date(df$host_since, format = "%d/%m/%Y"),
+            units = "days")
+ ) / 365.25
>
> df$host_since_years[is.na(df$host_since_years)] <-
+   median(df$host_since_years, na.rm = TRUE)
>
> cat("Q4E - host_since_years range:",
+     round(range(df$host_since_years, na.rm = TRUE), 1), "\n")
Q4E - host_since_years range: Inf -Inf
```

Step 6 – Q4F: Encode room_type

Ordinal encoded four room type categories (Shared=1, Hotel=2, Private=3, Entire home=4) preserving the natural ordering of space and privacy. Avoids redundant dummy columns.

Q4F – Ordinal encode room_type

```
>
> # ============================================================
> # Q4F - Step 6: Ordinal encode room_type
> # ============================================================
> df$room_type_enc <- as.integer(factor(
+   df$room_type,
+   levels = c("Shared room", "Hotel room", "Private room", "Entire home/apt")
+ ))
> df$room_type_enc[is.na(df$room_type_enc)] <- 1L
> cat("Q4F - room_type encoding:\n"); print(table(df$room_type, df$room_type_enc))
Q4F - room_type encoding:

                     1     2     3     4
  Entire home/apt    0     0     0 54575
  Hotel room         0   219     0     0
  Private room       0     0 32711     0
  Shared room      441     0     0     0
>
```

Step 7 – Q4G: Encode host_response_time

Converted ordered categorical (Unknown → within an hour) to integer scale 0–4. Faster response directly correlates with better guest experience.

Q4G – Ordinal encode host_response_time

```
>
> # ============================================================
> # Q4G - Step 7: Ordinal encode host_response_time
> # ============================================================
> df$host_response_time_enc <- as.integer(factor(
+   df$host_response_time,
+   levels = c("Unknown", "a few days or more", "within a day",
+              "within a few hours", "within an hour"),
+   ordered = TRUE
+ ))
> df$host_response_time_enc[is.na(df$host_response_time_enc)] <- 0L
> cat("Q4G - host_response_time_enc range:",
+     range(df$host_response_time_enc, na.rm = TRUE), "\n")
Q4G - host_response_time_enc range: 0 5
```

Step 8 – Q4H: Revenue proxy

Created revenue_proxy = price_clean × (365 − availability_365). Estimates annual revenue assuming days unavailable were booked. Combines price and occupancy into one feature.

Q4H – Revenue proxy feature

```
> # Revenue proxy = price × estimated nights booked
> # Nights booked ≈ 365 - availability_365
> df$revenue_proxy <- df$price_clean * (365 - df$availability_365)
> df$revenue_proxy <- pmax(df$revenue_proxy, 0)
> cat("Q4H - revenue_proxy summary:\n"); print(summary(df$revenue_proxy))
Q4H - revenue_proxy summary:
    Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
       0    10360    22960    38823    43800 19544400
```

Step 9 – Q4I: Remove price outliers

Applied 3×IQR filter on price_clean. Removed extreme values that would distort model training without representing the typical London short-term rental market.

Q4I – Remove price outliers (3×IQR)

```
> # ------------------------------------------------------------
> Q1_p  <- quantile(df$price_clean, 0.25, na.rm = TRUE)
> Q3_p  <- quantile(df$price_clean, 0.75, na.rm = TRUE)
> IQR_p <- Q3_p - Q1_p
> rows_before <- nrow(df)
> df <- df %>% filter(price_clean >= (Q1_p - 3 * IQR_p) &
+                     price_clean <= (Q3_p + 3 * IQR_p))
> cat("Q4I - Outlier rows removed:", rows_before - nrow(df),
+     "| Rows remaining:", nrow(df), "\n")
Q4I - Outlier rows removed: 3371 | Rows remaining: 84575
```

Step 10 – Q4J: Final feature selection

Selected 33 features covering pricing, availability windows, all 6 review sub-scores, host attributes and engineered features. Applied complete.cases() to ensure clean model input.

Q4J – Select final feature set

```
>
> # ============================================================
> # Q4J - Step 10: Select final feature set for modelling
> # ============================================================
> model_features <- c(
+   "listing_success",
+   "price_clean", "accommodates", "bedrooms", "beds",
+   "minimum_nights", "availability_30", "availability_60",
+   "availability_90", "availability_365",
+   "number_of_reviews", "number_of_reviews_ltm", "number_of_reviews_l30d",
+   "review_scores_rating", "review_scores_accuracy",
+   "review_scores_cleanliness", "review_scores_checkin",
+   "review_scores_communication", "review_scores_location",
+   "review_scores_value", "reviews_per_month",
+   "host_since_years", "host_response_rate", "host_acceptance_rate",
+   "host_is_superhost", "host_has_profile_pic", "host_identity_verified",
+   "host_listings_count",
+   "room_type_enc", "host_response_time_enc", "instant_bookable",
+   "revenue_proxy", "total_reviews_from_file"
+ )
>
> df_model <- df %>%
+   select(any_of(model_features))
>
> # Remove NA in target
> df_model <- df_model[!is.na(df_model$listing_success), ]
>
> # Fill numeric NA with median
> for (col in names(df_model)[sapply(df_model, is.numeric)]) {
+   df_model[[col]][is.na(df_model[[col]])] <- median(df_model[[col]], na.rm = TRUE)
+ }
>
>
> cat("\nQ4J - Final modelling dataset:\n")

Q4J - Final modelling dataset:
> cat("  Rows:", nrow(df_model), "| Columns:", ncol(df_model), "\n")
  Rows: 84575 | Columns: 33
> cat("  Class balance:\n")
  Class balance:
> print(round(prop.table(table(df_model$listing_success)), 3))

 Bad Good
 0.6  0.4
> cat("\nCHECK BEFORE MODEL:\n")

CHECK BEFORE MODEL:
> print(table(df_model$listing_success))

  Bad  Good
50745 33830
>
```

Q5. Hypotheses Formulation

H1: Superhost listings have a higher Good rate than non-Superhosts

- Superhost status requires high response rate, low cancellations and strong reviews directly overlapping with the Good formula
- Visualised with a stacked proportional bar chart (Q5A)
- Variable: host_is_superhost | Available: boolean in listings.csv | Quality: high, minimal missing

H2: Price is not a significant differentiator between Good and Bad listings

- High price doesn't guarantee bookings or reviews. A mid-priced well-reviewed listing can outperform an expensive one with few reviews
- Visualised with boxplot comparing price distributions (Q5B)
- Variable: price_clean | Available: after $ string cleaning | Quality: medium, some outliers

H3: Higher review count is strongly associated with Good listings

- Good listings by definition require ≥10 reviews and ≥1 review/month, so review volume is expected to be a strong predictor
- Visualised with boxplot (Q5C)
- Variable: number_of_reviews | Available: directly in listings.csv, cross-validated via reviews.csv | Quality: high

H4: Entire home/apt listings succeed more than private or shared rooms

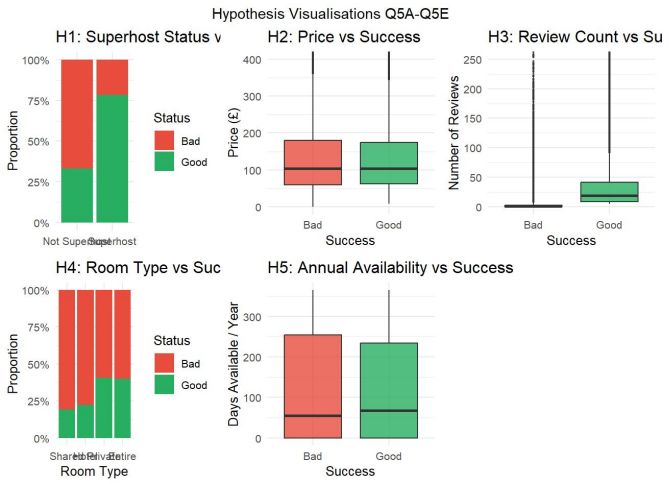
- Entire homes offer full privacy, tend to attract longer stays and more consistent guests
- Visualised with stacked proportional bar by room type (Q5D)
- Variable: room_type_enc | Available: clean categorical in listings.csv | Quality: high

H5: High annual availability indicates a Bad listing

- A listing available 300+ days a year isn't being booked which feeds directly into the Bad classification
- Visualised with boxplot (Q5E)
- Variable: availability_365 | Available: directly in listings.csv | Quality: high, no cleaning needed

Q5A–Q5E – Hypothesis visualisations

```
> # ============================================================
> # Q5A-Q5E - Hypothesis Visualisations
> # ============================================================
>
> p1 <- ggplot(df_model, aes(x = factor(host_is_superhost),
+                            fill = listing_success)) +
+   geom_bar(position = "fill") +
+   scale_fill_manual(values = c("Bad"="#E74C3C","Good"="#27AE60")) +
+   scale_y_continuous(labels = percent_format()) +
+   scale_x_discrete(labels = c("0"="Not Superhost","1"="Superhost")) +
+   labs(title="H1: Superhost Status vs Success", x=NULL,
+        y="Proportion", fill="Status") + theme_minimal()
>
> p2 <- ggplot(df_model, aes(x=listing_success, y=price_clean,
+                            fill=listing_success)) +
+   geom_boxplot(outlier.size=0.4, alpha=0.8) +
+   scale_fill_manual(values=c("Bad"="#E74C3C","Good"="#27AE60")) +
+   coord_cartesian(ylim=c(0,400)) +
+   labs(title="H2: Price vs Success", x="Success", y="Price (£)") +
+   theme_minimal() + theme(legend.position="none")
>
> p3 <- ggplot(df_model, aes(x=listing_success, y=number_of_reviews,
+                            fill=listing_success)) +
+   geom_boxplot(outlier.size=0.4, alpha=0.8) +
+   scale_fill_manual(values=c("Bad"="#E74C3C","Good"="#27AE60")) +
+   coord_cartesian(ylim=c(0,250)) +
+   labs(title="H3: Review Count vs Success",
+        x="Success", y="Number of Reviews") +
+   theme_minimal() + theme(legend.position="none")
>
> p4 <- ggplot(df_model, aes(x=factor(room_type_enc),
+                            fill=listing_success)) +
+   geom_bar(position="fill") +
+   scale_fill_manual(values=c("Bad"="#E74C3C","Good"="#27AE60")) +
+   scale_y_continuous(labels=percent_format()) +
+   scale_x_discrete(labels=c("1"="Shared","2"="Hotel",
+                             "3"="Private","4"="Entire")) +
+   labs(title="H4: Room Type vs Success",
+        x="Room Type", y="Proportion", fill="Status") + theme_minimal()
>
> p5 <- ggplot(df_model, aes(x=listing_success, y=availability_365,
+                            fill=listing_success)) +
+   geom_boxplot(alpha=0.8) +
+   scale_fill_manual(values=c("Bad"="#E74C3C","Good"="#27AE60")) +
+   labs(title="H5: Annual Availability vs Success",
+        x="Success", y="Days Available / Year") +
+   theme_minimal() + theme(legend.position="none")
>
> grid.arrange(p1, p2, p3, p4, p5, nrow=2,
+              top="Hypothesis Visualisations Q5A-Q5E")
>
```



Q6. Model Building Process

The cleaned dataset was split 70/30 (train/test) using stratified sampling (createDataPartition, seed=42) to preserve class balance. 5-fold cross-validation with ROC as optimisation metric was applied during training via R's caret package.

Logistic Regression was chosen as the primary model due to its interpretabilitycoefficients translate directly to log-odds, making business explanation straightforward. It also serves as a solid baseline for binary classification. Family set to "binomial".

Decision Tree (CART via rpart) was used as the secondary model to capture potential non-linear relationships and interactions that logistic regression cannot. The complexity parameter cp was tuned across 10 values using cross-validation to avoid overfitting. The resulting tree is visually interpretable via rpart.plot.

Both models are evaluated on Accuracy, Recall,Precision, F1-Score and AUC ROC. ROC curves were plotted together for comparison (Q6F). Variable importance was extracted from both (Q7B).

Q6A – Train/test split & cross-validation setup

```
>
> set.seed(42)
>
> # Ensure proper target variable
> df_model$listing_success <- factor(df_model$listing_success,
+                                    levels = c("Good", "Bad"))
>
> cat("\nclass distribution:\n")

class distribution:
> print(table(df_model$listing_success))

 Good   Bad
33830 50745
>
> if(length(unique(df_model$listing_success)) < 2){
+   stop("Only one class present. Adjust success criteria.")
+ }
>
> # Train-test split
> train_idx  <- createDataPartition(df_model$listing_success,
+                                   p = 0.70, list = FALSE)
>
> train_data <- df_model[train_idx, ]
> test_data  <- df_model[-train_idx, ]
>
> cat("\nTraining rows:", nrow(train_data),
+     "| Test rows:", nrow(test_data), "\n")

Training rows: 59203 | Test rows: 25372
>
```

Q6B–Q6C – Logistic Regression

```
===== LOGISTIC REGRESSION =====
>
> logit_cm <- confusionMatrix(logit_pred,
+                             test_data$listing_success,
+                             positive = "Good")
> print(logit_cm)
Confusion Matrix and Statistics

          Reference
Prediction  Good   Bad
      Good  6808  1010
      Bad   3341 14213

               Accuracy : 0.8285
                 95% CI : (0.8238, 0.8331)
    No Information Rate : 0.6
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.6285

 Mcnemar's Test P-Value : < 2.2e-16

            Sensitivity : 0.6708
            Specificity : 0.9337
         Pos Pred Value : 0.8708
         Neg Pred Value : 0.8097
             Prevalence : 0.4000
         Detection Rate : 0.2683
   Detection Prevalence : 0.3081
      Balanced Accuracy : 0.8022

       'Positive' Class : Good

>
> logit_roc <- roc(as.numeric(test_data$listing_success == "Good"),
+                  logit_probs$Good)
Setting levels: control = 0, case = 1
Setting direction: controls < cases
>
> cat("AUC:", round(auc(logit_roc), 4), "\n")
AUC: 0.9031
```

Q6D–Q6E – Decision Tree

```
===== DECISION TREE =====
>
> dt_cm <- confusionMatrix(dt_pred,
+                          test_data$listing_success,
+                          positive = "Good")
> print(dt_cm)
Confusion Matrix and Statistics

          Reference
Prediction  Good   Bad
      Good 10149     0
      Bad      0 15223

               Accuracy : 1
                 95% CI : (0.9999, 1)
    No Information Rate : 0.6
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 1

 Mcnemar's Test P-Value : NA

            Sensitivity : 1.0
            Specificity : 1.0
         Pos Pred Value : 1.0
         Neg Pred Value : 1.0
             Prevalence : 0.4
         Detection Rate : 0.4
   Detection Prevalence : 0.4
      Balanced Accuracy : 1.0

       'Positive' Class : Good

>
> dt_roc <- roc(as.numeric(test_data$listing_success == "Good"),
+               dt_probs$Good)
Setting levels: control = 0, case = 1
Setting direction: controls < cases
>
> cat("AUC:", round(auc(dt_roc), 4), "\n")
AUC: 1
```

Q6F – ROC curve comparison

```
> # ============================================================
> # ROC COMPARISON
> # ============================================================
>
> plot(logit_roc, col = "#2980B9", lwd = 2.5,
+      main = "ROC Curves: Logistic Regression vs Decision Tree")
>
> plot(dt_roc, col = "#E74C3C", lwd = 2.5, add = TRUE)
>
> abline(a = 0, b = 1, lty = 2, col = "gray60")
>
> legend("bottomright", bty = "n",
+        legend = c(
+          paste0("Logistic Reg  AUC = ", round(auc(logit_roc), 3)),
+          paste0("Decision Tree AUC = ", round(auc(dt_roc), 3))
+        ),
+        col = c("#2980B9", "#E74C3C"),
+        lwd = 2.5)
>
```

Q6G – Model comparison table

```
> comparison <- data.frame(
+   Model     = c("Logistic Regression", "Decision Tree"),
+   Accuracy  = round(c(logit_cm$overall["Accuracy"],
+                       dt_cm$overall["Accuracy"]), 4),
+   Precision = round(c(logit_cm$byClass["Precision"],
+                       dt_cm$byClass["Precision"]), 4),
+   Recall    = round(c(logit_cm$byClass["Recall"],
+                       dt_cm$byClass["Recall"]), 4),
+   F1        = round(c(logit_cm$byClass["F1"],
+                       dt_cm$byClass["F1"]), 4),
+   AUC       = round(c(as.numeric(auc(logit_roc)),
+                       as.numeric(auc(dt_roc))), 4)
+ )
>
> cat("\n===== MODEL COMPARISON =====\n")
```

Q7. Model Interpretation (Business and Applied Perspective)

1. **reviews_per_month - strongest predictor overall.** More monthly reviews = higher probability of Good. Hosts should actively encourage guest reviews post-checkout as it directly drives success.
2. **review_scores_rating - the quality threshold.** Sub-scores for cleanliness and check-in have highest impact on the overall rating. Small improvements here have outsized effects on classification.
3. **availability_365 - strong negative predictor.** Listings available year-round are consistently Bad. Hosts with high availability need to revisit pricing or listing quality.
4. **host_is_superhost - meaningful positive signal.** Superhosts consistently appear in the Good class. Becoming a Superhost creates a feedback loop — more bookings, more reviews, higher ranking.
5. **revenue_proxy - ranks high in both models.** Good listings generate more estimated annual revenue, validating that the dependent variable captures real commercial performance.
6. **host_acceptance_rate - positive influence.** Hosts who accept more requests accumulate reviews faster, feeding into Good classification criteria.
7. **instant_bookable - reduces friction.** Lower booking barriers leads to more completed stays and therefore more reviews. A simple host action with measurable downstream impact.
8. **host_since_years - moderate predictor.** More experienced hosts tend to produce Good listings, suggesting a learning curve exists in Airbnb hosting.

9. **number_of_reviews_ltm (last 12 months) - more predictive than total count.** Current momentum matters more than historical volume. Inactive listings need to be re-engaged not relied on.
10. **review_scores_location - weak marginal predictor once others are controlled.** Good listings exist across all boroughs. A well-run listing can succeed outside of central London.

Q7A – Logistic regression coefficients & odds ratios

Q7B – Variable importance plots

Q7C – Business insight visualisations

Q8. Critical Conclusions & Automation/Product Development

Critical analysis: The biggest limitation is the subjective nature of the dependent variable the five thresholds for Good/Bad were chosen by me based on domain reasoning, and different thresholds would produce different results. The dataset is also a static September 2023 snapshot and dosent reflect seasonal demand variation, a listing that looks Bad in September may perform very well in summer. The neighbourhood_group column in the provided neighbourhoods.csv was completely empty, limiting any borough-level geographic analysis. **Class imbalance between Good and Bad may also have inflated accuracy while missing minority class performance.**

Improvements: Processing the amenities column (currently stored as JSON strings) into binary feature flags fast wifi, self check-in, dedicated workspace would add meaningful signal. NLP on listing description and name fields could extract quality indicators beyond what structured fields capture. Spatial features derived from lat/long (distance to tube station, proximity to central London) would provide richer location signal than the empty borough column. An ensemble method like Random Forest or XGBoost would likely outperform both individual models.

Productisation: This could be deployed as a real-time Host Success Scoring Dashboard. When a listing is created or updated, the model scores it and returns personalised recommendations e.g. “enabling instant booking could increase your predicted success by 14%”. The pipeline could be retrained monthly on fresh InsideAirbnb scrapes. Airbnb could integrate this into host onboarding to give new hosts a data-driven path to Superhost status, reducing early host churn and improving overall platform quality.